🔎 Repo Exploration & Task Scope

Repo Exploration & Task Scope

Repo exploration is where the task becomes specific. The goal is to find a realistic engineering question that requires codebase investigation and repo-specific evidence.

Repo Exploration

Exploration Process

Work from broad context toward a narrow, answerable question.

① **Map the repo area.**

Find the entry points, important modules, tests, configs, and files tied to the assigned intent. Build a short mental model before writing anything.

② **Trace the relevant behavior.**

Follow call chains, state updates, route transitions, config loading, data flow, PR dependencies, or error paths. Capture the exact files and functions that support the eventual golden answer.

③ **Choose a task-shaped question.**

The question should be narrow enough to answer definitively, but it should require meaningful exploration. A good task often asks about a non-obvious interaction, indirect dependency, lifecycle ordering, blast radius, or causal chain.

④ **Check whether the task is fair.**

A reviewer should be able to verify the answer from the repository, patch, or executable evidence. Hidden context or guesswork means the task needs revision before proceeding.

Intent Guide

Use the assigned intent to decide what kind of question to ask.

Intent	Definition	Examples
Architecture & System Design	How components are structured, interact, and why design decisions were made.	"How does authentication flow through this middleware stack?" "We need to add a caching layer to this service... how should we design it?"
Root Cause Analysis	Why behavior is broken, unexpected, or failing, traced through the codebase. These tasks include pre-written `pr.patch` files.	"This function returns None intermittently but I can't reproduce it locally. What's happening?" "Why does this test pass in isolation but fail in the suite?"
Code Onboarding & Comprehension	Plain-language walkthrough of what existing code, files, or functions do.	"I just joined this project. What does this module actually do?" "What is the entry point for this service, and how does execution flow from there?"
PR Triage & Impact Assessment	Explain a code change, assess blast radius, classify severity/complexity, and flag security risk. These tasks include pre-written `pr.patch` files.	"What does this PR actually change, and is anything likely to break?" "This diff touches the auth module. What are the security implications?" Real case: a PR adds a tag-parsing helper to an analytics schema. Use `pr.patch` plus the existing view that references the function to explain what changes, what could break, severity, and security concerns.

Task Scope

Scope Guidelines

These guidelines describe the line between a strong repo-grounded task and a generic or unfair task.

Guideline	Description	Examples / Tips
Ground the task in repo evidence	The question should depend on concrete files, functions, configs, routes, tests, state transitions, or patch context.	Example evidence: mini-cart handlers + route definitions. Example evidence: composition, transform interface, bbox/keypoint utilities. Example evidence: `pr.patch` + migration code impact.
Make the reasoning non-local	The model should connect multiple pieces of evidence or understand an interaction.	Good task shapes: handler vs route behavior; processor setup vs call-time validation; patch change vs downstream blast radius.
Keep the prompt fair	Give enough starting context for the model to know what problem to investigate.	Good: observed reload symptom + named UI actions. Bad: “Something is wrong with navigation. Find it.”
Avoid lookup tasks	Tasks need more than a single obvious function, docstring, or grep result.	Bad: “What does `parseTags` return?” Better: “How does the parser affect downstream analytics behavior?”
Avoid artificial vagueness	Hard tasks should be hard because repo behavior is subtle. The prompt still needs enough context to be fair.	If a reviewer cannot tell what evidence should answer it, the task is underspecified.
Check uniqueness	The task should be meaningfully distinct from prior submissions.	A unique task needs more than changed repo/file names in a repeated prompt pattern.

Scope Examples

Use examples like these to calibrate how much context belongs in the task idea.

Type	Example	Commentary
✅ **Good root cause scope**	Mini-cart reload example: “Trace which files and handlers implement Edit shopping bag / Proceed to checkout, what API they use to change screens, and how that differs from SPA navigation.”	Specific observed behavior. Requires multiple files. Does not reveal the `window.location.href` conclusion.
❌ **Bad root cause scope**	“The bug is caused by `window.location.href` in miniCart.js. Explain why.”	Gives away the answer path. Weakens exploration and difficulty.
✅ **Good onboarding scope**	`Compose` flow example: Ask how processors, accepted keys, aliases, validation, child transforms, filtering, and postprocess fit together.	A real onboarding question. Requires tracing across several modules.
❌ **Bad onboarding scope**	“What is `Compose`?”	Definition-style lookup. Insufficient repo-specific reasoning.
✅ **Good PR / architecture scope**	PR / architecture example: Ask what ripples from a patch, or where a time-based autoscaling feature should fit in existing architecture.	Requires blast-radius or design reasoning beyond summary.
❌ **Bad PR / architecture scope**	“Summarize this diff” or “write a PRD for autoscaling.”	Too shallow or too detached from current repo structure.